Dense networks & backpropagation

Based on all 10 questions in the DTU written exam, 10 December 2025. Worked answers are derived here, not an official answer key. These notes cover this paper, not the full course syllabus.

Q1–2 · Count connections; track dimensions

For a dense layer with d inputs and h outputs, each output has d weights and one bias. Parameters = $dh + h = (d + 1)h$. Sum over layers; input nodes, ReLU and softmax add no trainable parameters. Batch size never changes the count.

> Q1: architecture $2 \to 4 \to 3$.
> Weights: $2 \times 4 + 4 \times 3 = 20$. Biases: $4 + 3 = 7$.
> Total = 27 parameters → D.

With column vectors, $z = Wx + b$ uses W of shape [out, in]. For a batch X of shape [B, in], PyTorch computes $XW^T + b$ and returns [B, out]. `nn.Linear(in, out)` stores its weight as [out, in].

Q2: architecture $3 \to 4 \to 2$, so $W^{(1)}$ is 4×3 and $W^{(2)}$ is 2×4. Therefore $n = 4, m = 3, k = 2, \ell = 4$ → B. Check: $(2 \times 4)(4 \times 1)$ produces a 2×1 output.

Q3 · Forward first, multiply local derivatives

Given $z = wx$, $h = \mathrm{ReLU}(z)$, $y = vh$ and $L = \tfrac{1}{2}(y - t)^2$:

> $\partial L/\partial w = (y - t) \cdot v \cdot 1[z > 0] \cdot x$
> $\partial L/\partial v = (y - t) \cdot h$

Q3: $x = 1$, $t = 6$, $w = 2$, $v = 4$. Forward: $z = 2$, $h = 2$, $y = 8$, $L = 2$. Backward: $(8 - 6) \times 4 \times 1 \times 1 = 8$ → C.

$\mathrm{ReLU}'(z) = 1$ for $z > 0$ and 0 for $z < 0$; at $z = 0$ it is not differentiable (PyTorch uses 0). An inactive ReLU blocks the gradient along that path. At a branch, add gradient contributions from every downstream path.

Traps: forgetting v or x; differentiating before the forward pass; keeping a factor of 2 despite the ½ in the loss. For $L = (y - t)^2$ without ½, the gradient doubles. Gradient descent uses $w \leftarrow w - \eta\, \partial L/\partial w$.

PyTorch outputs & losses

Q4 · Flatten images without flattening the batch

Input [B, 3, 10, 10] has 300 features per image. Required path: [B, 300] → [B, 128] → [B, 128] → [B, 5].

```
model = nn.Sequential(
    nn.Flatten(),                 # start_dim=1: keep B
    nn.Linear(300, 128), nn.ReLU(),
    nn.Linear(128, 128), nn.ReLU(),
    nn.Linear(128, 5), nn.Softmax(dim=1)
)
```

Q4 → D. Linear acts on the last dimension; it does not flatten images automatically. `Flatten(128)` means "start at dimension 128", not "make 128 features".

Q7 · Cross-entropy takes logits, not probabilities

For C classes: model output [B, C]; integer targets [B] with dtype `torch.long` and values 0,…,C−1. Cross-entropy combines log-softmax and negative log-likelihood:

$L = -\log \text{softmax}(z)_t = -z_t + \log \sum_j \exp(z_j)$

```
model = nn.Sequential(nn.Linear(10, 32), nn.ReLU(),
                      nn.Linear(32, 3))
loss_fn = nn.CrossEntropyLoss()  # Q7 → A
```

Q4 includes softmax because it asks for a classifier, not its training loss. When using CrossEntropyLoss, remove that final softmax. For inference, use `logits.softmax(dim=1)` ; `argmax(dim=1)` selects the same class with or without softmax.

Q9 · Predict an unrestricted mean and positive scale

Use 5 → 16 → 2 with ReLU only in the hidden layer. Interpret y[:,0] as μ and y[:,1] as $s = \log \sigma$; then $\sigma = \exp(s) > 0$.

```
def gaussian_nll(y, target):
    mu, s = y[:, 0], y[:, 1]       # both [B]
    return (s + 0.5 * ((target-mu)*torch.exp(-s))**2).mean()
```

Q9 → D: $L = \log \sigma + \tfrac{1}{2}((t - \mu)/\sigma)^2$, ignoring a constant. A predicts only a mean; B applies Softplus to both outputs, incorrectly restricting the mean to positive values for a general Gaussian; C permits non-positive σ.

Watch: $s = \log \sigma$ is not log variance. For $r = \log \sigma^2$, use $\tfrac{1}{2}[r + (t - \mu)^2 \exp(-r)]$. Match target and prediction shapes: [B,1] − [B] can silently broadcast to [B,B].

Convolutions & attention

Q5 · Slide, multiply, sum

A standard deep-learning convolution uses cross-correlation: do not flip the kernel. Each output sums elementwise patch × kernel products across spatial positions and input channels; then adds one bias if enabled.

```
X = [[1, 2, 0, 1],       K = [[1, 0],
     [1, 0, 1, 0],            [0, 1]]
     [1, 0, 2, 2],
     [0, 2, 1, 1]]
```

Q5: stride 2, padding 0, no bias. Four 2 × 2 patches start at (row, column) = (0,0), (0,2), (2,0), (2,2). K adds each patch's diagonal: 1 + 0, 0 + 0, 1 + 2, 2 + 1.

Output = [[1, 0], [3, 3]] → C.

Q6 · Output shape and parameter count

> For either spatial axis:
> $\text{size}_{\text{out}} = \text{floor}((\text{size}_{\text{in}} + 2p - d(k - 1) - 1) / s) + 1$
> p = padding per side; d = dilation (default 1); k = kernel size; s = stride.

Q6: input [5,5,5], 10 kernels of shape [5,3,3], p = 1, s = 2. Spatial size = floor((5 + 2 − 3)/2) + 1 = 3. Output channels equal the number of kernels: [10,3,3] → B.

For ordinary convolution (groups = 1), parameters = $C_{\text{out}}C_{\text{in}}k_H k_W + C_{\text{out}}$ biases if enabled. Q6 has no bias: 10 × 5 × 3 × 3 = 450. Spatial output positions reuse weights; they do not multiply the parameter count.

Q8 · Attention is a weighted average of values

Q: $[N_q, d_k]$, K: $[N_k, d_k]$, V: $[N_k, d_v]$. Scores QK^T have shape $[N_q, N_k]$. Apply softmax across keys within each row, then multiply by V; output shape is $[N_q, d_v]$.

```
Q = [[1,1,0], [0,0,1]]   K = [[1,0,0], [1,0,0]]
V = [[2,0,6,0], [0,4,0,8]]
Q @ K.T = [[1,1], [0,0]]
softmax(scores, dim=-1) = [[0.5,0.5], [0.5,0.5]]
A = [[1,2,3,4], [1,2,3,4]]                    # Q8 → C
```

Equal scores give uniform weights—even all-zero scores. Each row sums to 1. This exam uses non-scaled attention; standard scaled attention uses $QK^T/\sqrt{d_k}$. Softmax is $\exp(z_i)/\sum \exp(z_j)$; subtracting the row maximum improves numerical stability without changing it.

Sequence models & final checks

Q10 · Many-to-one means one prediction per sequence

For unbatched one-hot input [T,V], `nn.LSTM(V,32)` returns `out` of shape [T,32] and `(h_n,c_n)`, each [1,32] for one layer and one direction. Use the final time step, then project 32 hidden features to 3 logits.

```
class MyLSTM(nn.Module):
    def __init__(self, V):
        super().__init__()
        self.lstm = nn.LSTM(V, 32)
        self.linear = nn.Linear(32, 3)

    def forward(self, x):
        out, (h_n, c_n) = self.lstm(x)
        return self.linear(out[-1])  # [32] → [3]
```

Q10 → A. `out[-1]` selects the final step; `out[:-1]` selects all but the final step. The LSTM module itself cannot be indexed as a sequence. Returning only out[-1] leaves 32 features, not 3 class logits.

For a portable loss call with the exam's target [1], explicitly batch the logits: `loss_fn(model(x).unsqueeze(0), target)`, giving [1,3] and [1]. The conventional unbatched pairing is logits [3] and a scalar class target.

With batches, the default LSTM input is [T,B,V]; `batch_first=True` uses [B,T,V], so select `out[:, -1, :]`. For padded unequal-length sequences, the last array position may be padding, not the last valid token.

Worked answer map

Question	Choice	Decisive check
1 / 2	D / B	27 parameters / W shapes 4×3 and 2×4
3 / 4	C / D	Gradient 8 / flatten to 300 features
5 / 6	C / B	[[1,0],[3,3]] / output [10,3,3]
7 / 8	A / C	Raw logits + cross-entropy / uniform averaging
9 / 10	D / A	Log standard deviation / final state + classifier

BEFORE CHOOSING AN ANSWER

Trace every tensor shape → count weights and biases separately → check loss factors → check output constraints → check the reduction axis. Compute the result before matching a multiple-choice option.

Source: 02456_exam_2025.pdf, pp. 1–5. Question references follow the supplied paper. "Don't know" and scoring rules depend on the separate answer sheet, which was not supplied.